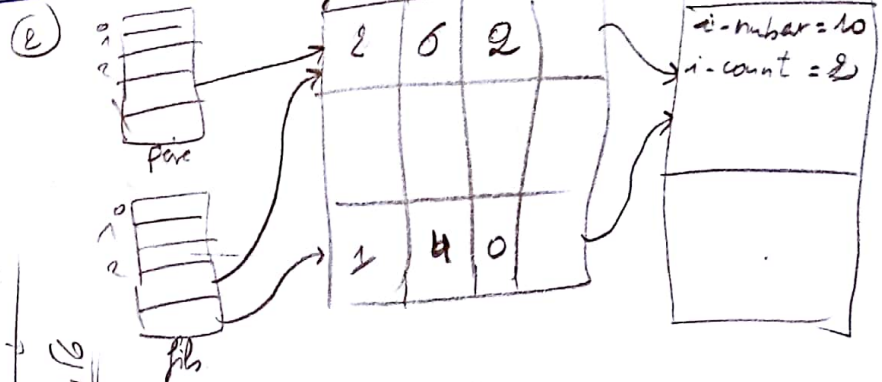


Examen 2017.

flag {
0 → lecture
2 → écriture
2 → lecture/écriture

48, 35
7/6



M1 - Noyau 4I401

Janvier 2017

2 heures - Tout document papier autorisé

Barème sur 21 donné à titre indicatif

L. Arantes, P. Cadinot, E. Encrenaz, J. Lejeune, P. Sens, J. Sopena

I. FICHIERS (7 POINTS)

Nous considérons une partition de numéro 0 (dev = 0) organisée de la manière suivante : un *boot bloc* occupe le bloc 0, un *super-bloc* occupe le bloc 1, les blocs suivants contiennent les *inodes* suivis des blocs de données. Les *inodes* sont numérotées de 1 à 1000. Les blocs font 512 octets et les inodes 128 octets. Enfin, nous considérons que les inodes ont une structure équivalente à celle vue en TD.

I.1. (0,5 point)

Quel est le dernier numéro de blocs stockant des inodes (justifiez) ?

Soit le fichier `fich1.txt` dont le contenu est le suivant :

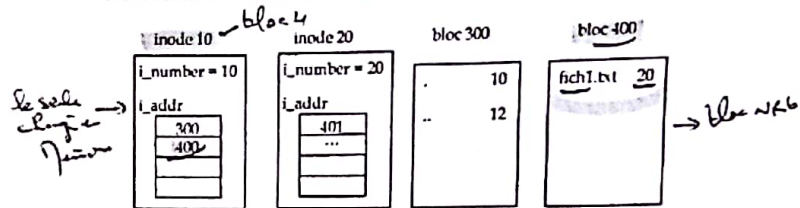
"Noyaux2017"

Soit le code suivant :

```
00 : int f1, f2;
01 : char buffer[10];
02 : f1 = open("../fich1.txt", O_RDWR);
03 : buffer[4] = '\0';
04 : if (fork() == 0) {
05 :     write(f1, "Examen", 6);
06 :     int f2 = open("../fich1.txt", O_RDONLY); → ouvrir
07 :     read(f2, buffer, 4);
08 :     printf("is", buffer);
09 :     close(f1); close(f2);
10 :     exit(0);
11 : } else {
12 :     wait(NULL);
13 :     read(f1, buffer, 4);
14 :     printf("is", buffer);
15 :     close(f1);
16 :     exit(0);
17 : }
```

1/10

Nous considérons les inodes et les blocs suivants :



Le bloc 300 ne contient pas la chaîne de caractère "fich1.txt"

I.2. (1 point)

Quels sont les affichages produits par chacun des processus (justifiez) ?

I.3. (2 points)

Donnez l'état, juste après la ligne 7, des tables de descripteurs de fichiers, fichiers ouverts, et inodes (faites un schéma, pensez à représenter notamment les champs `f_count`, `f_offset`, `f_flags`, `i_count` et `i_number`).

Avant l'exécution de ce code, nous faisons l'hypothèse que seule l'inode du répertoire courant (inode numéro 10) est dans la table `inode[]`

I.4. (0,5 point)

Dans quel bloc disque se situe l'inode 20 (préciser la formule utilisée) ?

I.5. (1,5 points)

Donnez la séquence des blocs et inodes accédés jusqu'à la fin de la ligne 7. Pour chaque accès, spécifiez la ligne de code associée ainsi que les actions/fonctions internes exécutées qui induisent ces accès.

Soit la configuration suivante du buffer cache avant l'exécution du code. Le buffer cache ne peut contenir au maximum que 4 blocs. Les numéros de bloc sont représentés sous la forme `devno-blockno`. La fonction de hachage associant un bloc à une des sous-listes est `devno+blockno % 5`.

2/10

Noyaux 2017 → contenu du fichier

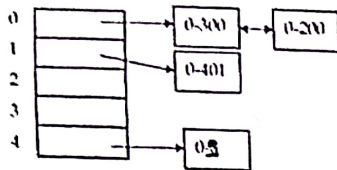
le fils écrit "Examen" ⇒ Examen 2017. → nouveau contenu du fichier.

le fils lit EXAM

le père lit EXAM

inode 20: $(N_{dev} + N_{inode}) / 4 \Rightarrow 0 + 20 / 4 = 5 + 1 (\text{super bloc}).$

Table de la machine



La bfreelist est la suivante (de la tête vers la queue) : 0-300, 0-401, 0-200, 0-3. Initialement aucun des blocs n'est marqué avec le flag DELWRI. On considérera que les écritures ne sont pas différées.

I.6. (1,5 points)

Donnez la configuration du buffer cache après la ligne 5 du code. Quel est le nombre d'entrées/sorties physiques ?

II. SYNCHRONISATION (4 POINTS)

Dans cet exercice, on veut ajouter au noyau la possibilité d'attendre la fin d'une entrée/sortie (E/S) asynchrone via une fonction interne fin_ES.

II.1. (2,5 points)

Donnez l'implémentation d'une fonction noyau "fin_ES(devno, blkno)" qui bloque un processus si une E/S asynchrone est en cours et ce jusqu'à la fin de cette E/S.

II.2. (1,5 points)

Modifier le code original du noyau pour réveiller un processus en attente dans la fonction fin_ES. Vous indiquerez clairement la/les fonction(s) ou structure(s) modifiée(s), ainsi que la modification.

Attention : votre solution devra éviter toute perte de temps si aucun processus n'est en attente sur cet événement.

III. MONTAGE D'UN SYSTÈME DE FICHIERS (10 POINTS)

On s'intéresse dans cet exercice au montage d'un système de fichiers (ou partition) sur une arborescence existante. C'est le cas lorsque par exemple vous enfichez une clé USB sur votre ordinateur : l'arborescence du système de

fichiers contenu sur la clé est insérée dans l'arborescence actuellement déployée sur votre ordinateur, à partir d'un point de montage, et vous pouvez accéder de façon (presque totalement) transparente aux fichiers de l'arborescence initiale et de son extension.

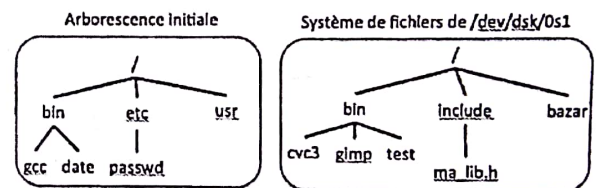
L'appel système `mount(fs_à_monter, point_de_montage, droits)` permet de réaliser ce montage.

On rappelle qu'un système de fichiers est structuré comme suit :

- bloc 0 - boot,
- bloc 1 - superbloc,
- blocs 2 à p - liste d'inodes,
- blocs p+1, ... - blocs de données et indirection des fichiers.

L'inode associée à la racine d'une arborescence est désigné par ROOTINO.

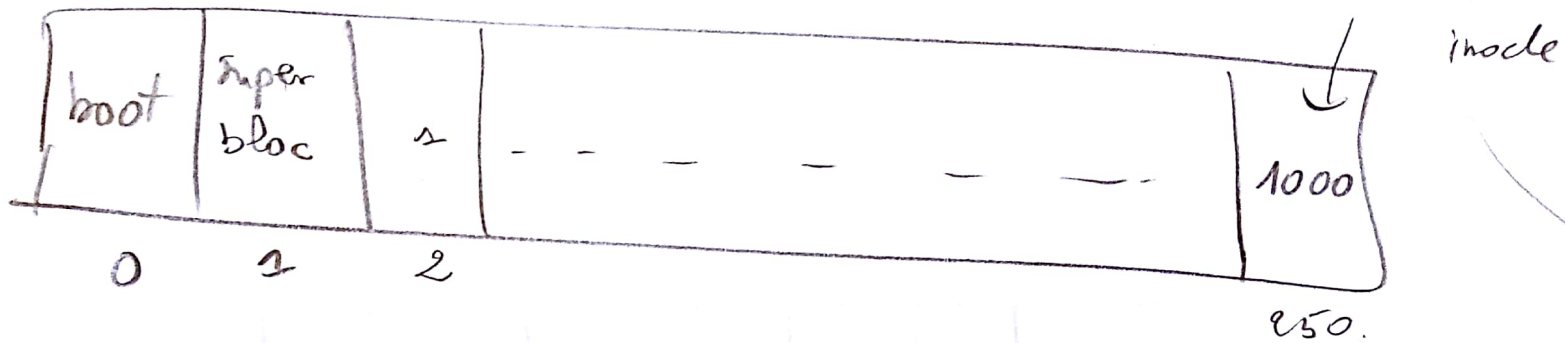
On suppose que le système contient une arborescence initiale décrite sur la figure à gauche. On dispose d'un périphérique amovible, nommé /dev/dsk/0s1, contenant le système de fichier décrit sur la droite de la figure.



La commande : `mount /dev/dsk/0s1 /usr` réalise le montage de l'arborescence de /dev/dsk/0s1 sur l'arborescence initiale, au point de montage /usr. On ne se préoccupe pas des droits d'accès.

Le résultat du montage est représenté ci-contre.

EXAMEN/2017



$$\text{bloc} = 512 \text{ } \emptyset$$

$$\text{inode} = 128 \text{ } \emptyset$$

$$\text{1000} \text{ bloc} ; 4 \text{ inode dans bloc } 512 / 128 = 4$$

1000 → dernière inode.

1000 → 4 inode.

$$1000 \begin{array}{r} 4 \\ \hline 250 \end{array}$$

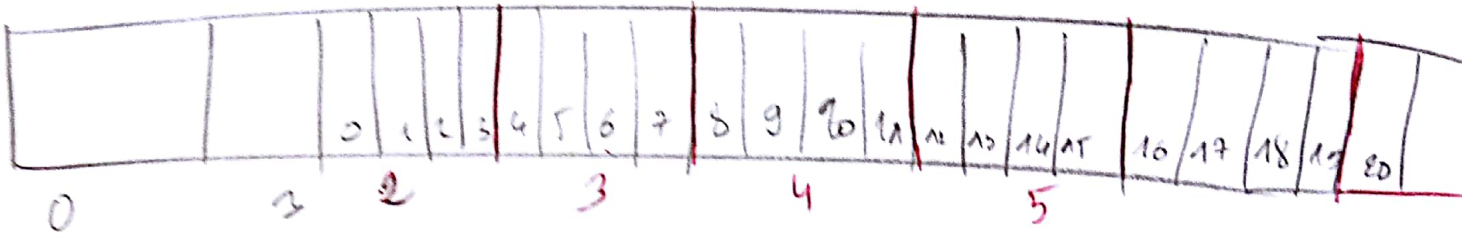
$$; - \text{Si on commence par } 0 = 251$$

$$(250 - 2) + 1 = 251$$

$$- \text{Si on commence par } 1 = 252$$

$$(252 - 2) + 1 = 252$$

$$\frac{20}{4} + 1 = 6.$$



$$\frac{\text{nr mode}}{4} + 1 = 6 \rightarrow 20.$$

① open: on accède à mode 10,
On parcourt 300 et 400.

2 E/S 300 et 400.

Open F₁.

Chercher mode N° 20.

1 E/S. → 20 (bloc 6).

• / fich 1.txt

⇒ chemin.

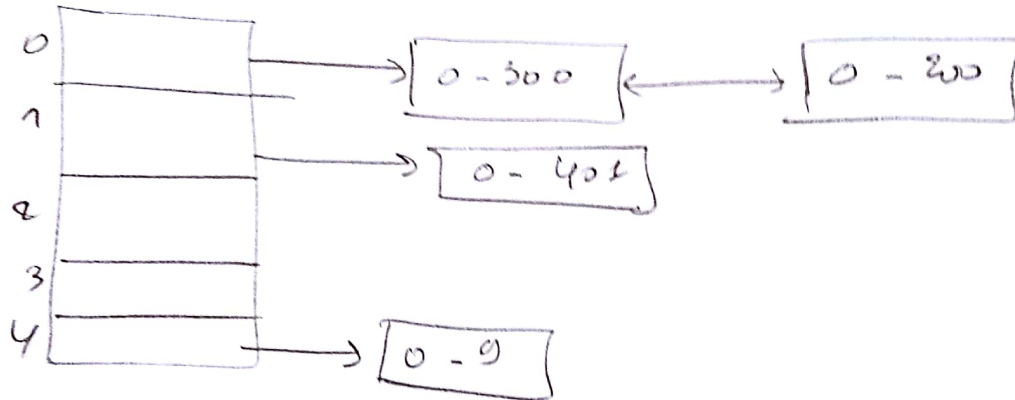
- write (F₁ . . .) ⇒ écrire dans 400

- Open 3 E/S

- read 0 E/S.

bfreelist : 0-300, 0-400, 0-200, 0-9.

ligne 1: Open (F₃) → 400 n'est pas dans le buffer



~~bfc = 0-400, 0-200, 0-9, 0-400 → 1 E/S.~~

~~write : 0-200, 0-9, 0-400, 0-300 → 1 E/S.~~

~~write : 0-200, 0-9, 0-300, 0-400 → 0 E/S.~~

~~2 E/S.~~

1er open ⇒ 0-400, 0-200, 0-9, 0-300 → ligne 3

⇒ 0-200, 0-9, 0-300, 0-400

⇒ 0-9, 0-300, 0-400, 0-6.

write ⇒ 0-300, 0-400, 0-6, 0-400.

ATTRAPAGE 2019:

-pid = 200

P-ppid = ..

P-pid = 300

P-ppid = 200

r-pid = 400

P-ppid = 300

① 300 fils de 200 ; 400 fils de 300.

200 → open en WRONLY
300 → open en WRONLY → ①

200 → open en RONLY
400 → open en RONLY → ②

400 → open en RDWR → ③

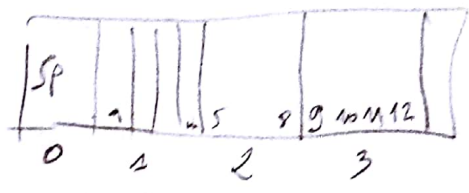
③ code :

```
int F1, F2; char buf[255];
F1 = open("noyau", O_WRONLY);
F2 = open("noyau", O_RDONLY);
write(F1, buf, 10);
if (fork == 0) { // fils pointe sur F1.
    if (fork == 0) {
        close(F1);
        F3 = open("noyau", O_RDWR);
        sleep(20);
        exit(0);
    }
    close(F2);
    wait(NULL);
}
```

Super bloc $\rightarrow 1$ bloc inode $\rightarrow 0 \text{ à } m$ bloc $\rightarrow 0 \dots n$

bloc = 512 oct. , inode = 128 octets.

inode 10 $\rightarrow$ courant



inode : 10 $\rightarrow$
 20 $\rightarrow$
 30 $\rightarrow$
 40 $\rightarrow$

~~10 + 12 = 22~~
~~10 + 12 = 22~~

$512 / 128 = 4$ inode per bloc.

~~10 $\rightarrow$ 1 (SB) + (inode - 1) / 4~~

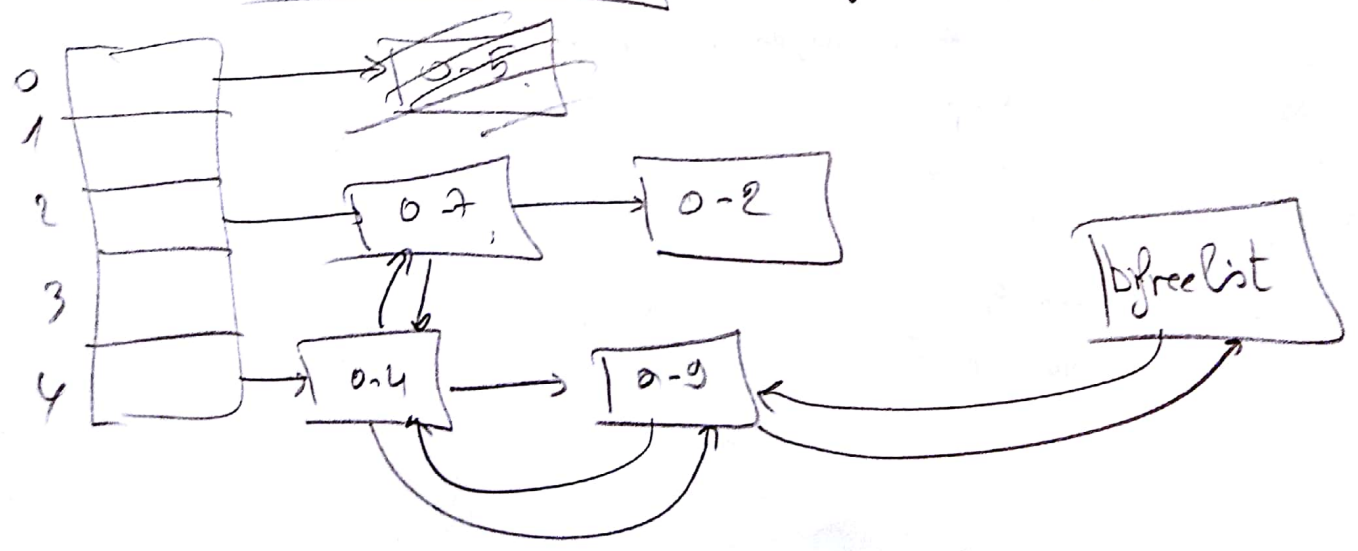
~~10 $\rightarrow$ 9 / 4 = 2 + 1 = 3~~

~~20 $\rightarrow$ 1 + (19 / 4) = 5~~

bfreelist: 0-5, 0-4, 0-3

0-4 : 0-5, 0-3, 0-4. bfreelist.

0-7 : [0-3, 0-4, 0-7] $\rightarrow$ bfreelist.



c) 1E/S.